CSC3002
C/C++ Programming

Template and Iterator

Prof. Dr. Ruizhi Chen， SDS

Office: TxC 717

E-mail: chenruizhi@cuhk.edu.cn

Template and Iterator

1 Templates
2 Implementing Iterator

Overloading

- In programming languages, ***polymorphism*** is the provision of a single interface to entities of different types or the use of a single symbol to represent multiple different types.
- Function/operator ***overloading*** (a form of polymorphism) allows you to define several functions with the same name as long as those functions can be distinguished by their signatures.
- Given a particular function call, the compiler can look at the number and types of the arguments (i.e., signature) and choose the version of the function that matches that signature.
- Overloaded functions can have quite different behaviors with different signatures; or simply different parameter types.

Exercise: Overloading

- What is the output?

```
#include <iostream>
using namespace std;

int operate(int a, int b)
{
   return a/b;
}

double operate(double a, double b)
{
   return a/b;
}

int main ()
{
   cout<<operate(1, 2)<<endl;
   cout<<operate(1.0, 2.0)<<endl;
   return 0;
}
```

For this type of overloading, C++ can help you generate the overloading automatically.

Templates

- One of the most powerful features in C++ is the ***template*** facility, which makes it possible to define functions and classes that work for a variety of types, e.g., to automate the overloading process in situations where the code is identical except for the types involved.
- The most common form of a template specification is

  `template <typename` *placeholder*`>`

  where *placeholder* is an identifier that is used to stand for a specific type when the definition following the **`template`** specification is compiled. The keyword **`typename`** can also be **`class`**. They are interchangeable in this context.
- For more than one type parameter, simply specify more template parameters between the angle brackets.

  `template <typename` *placeholder1*`,` **`typename`** *placeholder2*`>`

Templates in Functions

- Templates can be used before a function definition to create a generic collection of functions that can be applied to values of various types.
- The following code, for example, creates a template for the **Max** function, which returns the larger of its two arguments:

```
template <typename ValueType>
ValueType Max(ValueType v1, ValueType v2) {
   return (v1 > v2) ? v1 : v2;
}
```

- The compiler will generate the code for many different versions of **Max**, one for each type that the client uses.

ValueType = int or
ValueType = double

```
int Max(int v1, int v2) { return (v1 > v2) ? v1 : v2; }
double Max(double v1, double v2) { return (v1 > v2) ? v1 : v2; }
```

- Note that a template is not a class or a function. It is a "pattern" that the compiler uses to generate a family of classes or functions.

Templates in Functions

- The function **max** can be used only with types that implement the **>** operator. If you call **Max** on some type that doesn't, the compiler will signal an error.

```
Max('A', 'Z');
```

```
Max(Point(10,10), Point(10,100));
```

```
#include <string>
Max(std::string("cat"), std::string("dog"));
```

```
bool operator>(const Point &pt1, const Point &pt2);
bool operator>(const string &str1, const string &str2);

int main()
{
	string str1("abcdE"), str2("ABCDE");
	Point pt1(10, 10), pt2(1, 100);

	cout << Max(1.0, 1.1) << endl; // 1.1
	cout << Max('A', 'B') << endl; //'B'
	cout << Max(str1, str2) << endl; // str1
	Point pt = Max(pt1, pt2); // pt2(1,100)
	cout << "pt(" << pt.x << ", " << pt.y << ")" << endl;
	return 0;
}
```

Exercise: Multiple Types

- The following code creates a template for the **max** function, which returns the larger of its two arguments:

```
template <typename ValueType1, typename ValueType2>
ValueType1 Max(ValueType1 v1, ValueType2 v2) {
   return (v1 > v2) ? v1 : v2;
}
```

- What is the result of the following function call and why?

```
Max(1, 1.1) = 1;
```

Example: Rewrite `sort` as a Template

- Templates can be used before a function definition to create a generic collection of functions that can be applied to values of various types.
- Rewrite the following code so that it sorts any type that implements the < operator:

```
void sort(int array[], int n) {
   for (int i = 0; i < n; i++) {
      int minIndex = i;
      for (int j = i + 1; j < n; j++) {
         if (array[j] < array[minIndex]) minIndex = j;
      }
      swap(array[i], array[minIndex]);
   }
}

void swap(int & x, int & y) {
   int tmp = x;
   x = y;
   y = tmp;
}
```

Templates in Functions

- The following code, for example, creates a template for invoking selection sort on arrays of any element type:

```
template <typename ValueType>
void sort(ValueType *array, int n)
{
      for (int i = 0; i < n; i++)
      {
            int minIndex = i;
            for (int j = i + 1; j < n; j++)
            {
                  if (array[j] < array[minIndex])
                  minIndex = j;
            }
            swap(array[i], array[minIndex]);
    }
}

template <typename ValueType>
void swap(ValueType &x, ValueType &y)
{
            ValueType tmp = x;
            x = y;
            y = tmp;
}
```

Implementing the Template Class

- The first step in writing the template version of the **`Stack`** class is to add the **`template`** keyword to the interface just before the class definition:

```
template <typename ValueType>
class Stack {

   . . . body of the class . . .

};
```

- Once you have made this change, each instance of the specific type (formerly **`char`** in the **`CharStack`** class) must be replaced by the **`ValueType`** placeholder for the generic type, as in

```
ValueType *array;
```

or

```
void push(ValueType value);
```

Implementing the Template Class

- The final change necessary to implement the template class is to add **`template`** declarations to every method body, as in the following updated version of the constructor:

```
template <typename ValueType>
Stack<ValueType>::Stack() {
   capacity = INITIAL_CAPACITY;
   array = new ValueType[capacity];
   count = 0;
}
```

To instantiate a template class, you specify arguments for the template class enclosed by angle brackets after the class name wherever the class is used.

- Because of the restrictions that C++ imposes on template types, the implementations of the methods need to be included as part of the **`stack.h`** header, even though the details are not of interest to most clients.

Templates in Class Definitions

- Templates are more commonly used to define generic classes. When they are used in this way, the `template` keyword must appear before the class definition and before each of the implementations of the member functions.
- The most inconvenient aspect of using templates to create generic classes is that the compiler cannot process them correctly unless it has access to both the interface and the implementation at the same time. The effect of this restriction is that the .h files for template classes must contain *both* the prototypes and the corresponding implementation codes.
- To emphasize the conceptual separation between the interface and the associated implementation, you should make sure to include an appropriate comment before the private section and the implementation in the .h files warning casual clients away from the details.

The `stack.h` Interface

```
/*
 * File: stack.h
 * -------------
 * This interface exports a template version of the Stack class.  This
 * implementation uses a dynamic array to store the elements of the stack.
 */

#ifndef _stack_h
#define _stack_h

#include "error.h"

/*
 * Class: Stack<ValueType>
 * -----------------------
 * This class implements a stack of the specified value type.
 */

template <typename ValueType>
class Stack {

public:
```

The `stack.h` Interface

```
/*
 * Constructor: Stack
 * Usage: Stack<int> stack;
 * ------------------------
 * The constructor initializes a new empty stack containing
 * the specified value type.
 */

    Stack();

/*
 * Destructor: ~Stack
 * Usage: (usually implicit)
 * -------------------------
 * The destructor deallocates any heap storage associated
 * with this stack.
 */

    ~Stack();
```

The `stack.h` Interface

```
/* Comments for the simple methods have been elided for space */

    int size() const;

/* . . . */

    bool isEmpty() const;

/* . . . */

    void clear();

/* . . . */

    void push(ValueType value);

/* . . . */

    ValueType pop();

/* . . . */

    ValueType peek() const;

/* Copy constructor and assignment operator */

   Stack(const Stack<ValueType> & src);
   Stack<ValueType> & operator=(const Stack<ValueType> & src);
```

The `stack.h` Interface

```
/* Private section */

/*
 * Implementation notes
 * --------------------
 * This version of the stack.h interface uses a dynamic array to store
 * the elements of the stack.  The array begins with INITIAL_CAPACITY
 * elements and doubles the size whenever it runs out of space.  This
 * discipline guarantees that the push method has O(1) amortized cost.
 */

private:

   static const int INITIAL_CAPACITY = 10;

/* Instance variables */

   ValueType *array;                    /* A dynamic array of the elements */
   int capacity;                        /* The allocated size of the array */
   int count;                           /* The number of stack elements    */

/* Private method prototypes */

   void deepCopy(const Stack<ValueType> & src);
   void expandCapacity();
};
```

The `stack.h` Interface

```
/*
 * Implementation section
 * ----------------------
 * C++ requires that the implementation for a template class be available
 * to the compiler whenever that type is used.  The effect of this
 * restriction is that header files must include the implementation.
 * Clients should not need to look at any of the code beyond this point.
 */

/*
 * Implementation notes: Stack constructor and destructor
 * ------------------------------------------------------
 * These methods allocate and free the dynamic array.
 */

template <typename ValueType>
Stack<ValueType>::Stack() {
   capacity = INITIAL_CAPACITY;
   array = new ValueType[capacity];
   count = 0;
}

template <typename ValueType>
Stack<ValueType>::~Stack() {
   clear();
}
```

The `stack.h` Interface

```
/*
 * Implementation notes: size, isEmpty, clear
 * ------------------------------------------
 * These methods are generic forms of the versions used for CharStack.
 */

template <typename ValueType>
int Stack<ValueType>::size() const {
   return count;
}

template <typename ValueType>
bool Stack<ValueType>::isEmpty() const {
   return count == 0;
}

template <typename ValueType>
void Stack<ValueType>::clear() {
   if(array) delete[] array;
   array = NULL;
   count = 0;
}
```

The `stack.h` Interface

```
/*
 * Implementation notes: push, pop, peek
 * -------------------------------------
 * The push method tests to see whether the array needs to be expanded; pop
 * and peek must check for an empty stack.
 */

template <typename ValueType>
void Stack<ValueType>::push(ValueType value) {
   if (count == capacity) expandCapacity();
   array[count++] = value;
}

template <typename ValueType>
ValueType Stack<ValueType>::pop() {
   if (isEmpty()) cout << "pop: Attempting to pop an empty stack\n";
   return array[--count];
}

template <typename ValueType>
ValueType Stack<ValueType>::peek() const {
   if (isEmpty()) cout << "peek: Attempting to peek at an empty stack\n";
   return array[count - 1];
}
```

The `stack.h` Interface

```
/*
 * Implementation notes: copy constructor and assignment operator
 * --------------------------------------------------------------
 * These methods follow the standard template, leaving the work to deepCopy.
 */

template <typename ValueType>
Stack<ValueType>::Stack(const Stack<ValueType> & src) {
   deepCopy(src);
}

template <typename ValueType>
Stack<ValueType> & Stack<ValueType>::operator=(const Stack<ValueType> & src) {
   if (this != &src) {
     deepCopy(src);
   }
   return *this;
}
```

The `stack.h` Interface

```
/*
 * Implementation notes: deepCopy
 * ------------------------------
 * This function copies the data from the src parameter into the current
 * object.  All dynamic memory is reallocated to create a "deep copy" in
 * which the current object and the source object are independent.
 * The capacity is set so that the stack has some room to expand.
 */

template <typename ValueType>
void Stack<ValueType>::deepCopy(const Stack<ValueType> & src) {
   capacity = src.count + INITIAL_CAPACITY;
   clear();
   array = new ValueType[capacity];
   for (int i = 0; i < src.count; i++) {
      array[i] = src.array[i];
   }
   count = src.count;
}
```

The `stack.h` Interface

```
/*
 * Implementation notes: expandCapacity
 * ------------------------------------
 * This private method doubles the capacity of the elements array whenever
 * it runs out of space.  To do so, it copies the pointer to the old array,
 * allocates a new array with twice the capacity, copies the values from
 * the old array to the new one, and finally frees the old storage.
 */

template <typename ValueType>
void Stack<ValueType>::expandCapacity() {
   ValueType *oldArray = array;
   capacity *= 2;
   array = new ValueType[capacity];
   for (int i = 0; i < count; i++) {
      array[i] = oldArray[i];
   }
   delete[] oldArray;
}

#endif
```

The stack.h, another format

```
// Define the stack class
template <typename T>
class Stack
{
private:
      T *data; // Pointer to allocated array
      size_t capacity = 10; // Total capacity of the stack
      size_t size = 0; // Current number of elements

public:
      Stack(size_t initial_capacity = 10) // Constructor
      : capacity(initial_capacity), size(0)
      {
          data = new T[capacity];
      }

      T &pop()
      {
            if (get_size() > 0)
            return data[--size];
      }
.
.
.
}
```

Introduction to the C++ Libraries

- A collection of ***classes*** and ***functions***, which are written in the core language and part of the C++ ISO Standard itself. Features of the C++ Standard Library are declared within the `std` ***namespace***
  - Containers: vector, queue, stack, map, set, etc.
  - General: algorithm, functional, iterator, memory, etc.
  - Strings
  - Streams and Input/Output: iostream, fstream, sstream, etc.
  - Localization
  - Language support
  - Thread support library
  - Numerics library
  - C standard library: cmath, cctype, cstring, cstdio, cstdlib, etc.

Iterating over a collection

- One of the common operations that clients need to perform when using a collection is to iterate through the elements.
- While it is easy to implement iteration for vectors and grids using **`for`** loops, it is less clear how you would do the same for other collection types. The modern approach to solving this problem is to use a general tool called an ***iterator*** that delivers the elements of the collection, one at a time.
- C++11 uses a ***range-based for statement*** to simplify iterators:

```
for (string key : map) { . . . code to process that key . . . }
```

```
for (auto key : map) { . . . code to process that key . . . }
```

- ***Range-based for*** (provided since C++ 11) is a way to access ***iterators*** (provided by the implementors of the collections).

Using Iterators in C++

- When you work with strings, one of the most important patterns involves iterating through the characters in a string, which requires the following code (index-based loop):

```
for (int i = 0; i < str.length(); i++) {
    . . . Body of loop that manipulates str[i] . . .
}
```

- For collection classes, the range-based **for** loop looks like:

```
for (auto x : stk) {
    . . . Body of loop involving x . . .
}
```

- The C++ compiler translates the range-based **for** loop using ***iterators***, which is what you would do before C++11:

```
for (Stack::iterator it = stk.begin();
    it != stk.end(); it++) {
    . . . Body of loop involving *it . . .
}
```

Using Iterators in C++

- Every collection class in the STL exports an **`iterator`** type (a ***nested type***) along with two standard methods that produce iterators. The **`begin`** method returns an iterator positioned at the beginning of the collection. The **`end`** method returns an iterator positioned just past the final element.
- Iterators in C++ use a syntax derived from pointers. Given an iterator, one reads the corresponding value by dereferencing the iterator variable and increments it using the **`++`** operator. The pattern for using an iterator to loop over a collection **`c`** is:

```
for (ctype::iterator it = c.begin(); it != c.end(); it++) {
    . . . Body of loop involving *it . . .
}
```

Using Iterators in C++

The iterator class is a nested class of the container class allow to:

- access to the private member of the enclosing class (the container).
- prevent collisions and allow each container to have its own iterator.

Example: The `Vector` Iterator

- Some class may support only the **`InputIterator`** level of services (e.g. ++. -- operators), while the iterator for the **`Vector`** class is a **`RandomAccessIterator (e.g += operator)`**.

```
Vector<int>::iterator it = v.end();
while (it != v.begin()) {
   cout << *--it << endl;
}

for (Vector<int>::iterator it = v.begin(); it < v.end(); it += 2) {
   cout << *it << endl;
}
```

- Using iterators in such unusual ways often ends up making programs difficult to maintain, much like code that tries to be too clever with its pointer manipulation. The best way to specify iteration is to use the (range-based) **`for`** loop whenever you can, because doing so hides the iterators altogether.

Implementation of an iterator

- Iterators are considerably easier to implement for **`Vector`** (and, e.g., **`Grid`** and **`HashMap`**) than they are for most of the other collection classes.
- Implementing **`iterator`** for the **`Vector`** class presents a relatively straightforward approach, because the underlying structure of the vector is defined in terms of a simple dynamic array, and the only state information the iterator needs to maintain is the current index value, along with a pointer back to the **`Vector`** object itself.
- Iterators for tree-structured classes like **`Binary Search Tree`** turn out to be enormously tricky, mostly because the implementation has to translate the recursive structure of the data into an iterative form.
- As a general rule, it is wise to leave the implementation of iterators to experts.

Implementation of an iterator

See the example of a
Vector Class Implemented with
Template and Iterator

The End